

# **Competitive Programming**

**Section 21 Editorial: Graphs I Mixed Contest**

Competitive Programming Course

2026-07-05

# Table of contents

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Editorial overview</b>             | <b>4</b>  |
| <b>2</b> | <b>Tool reminders</b>                 | <b>5</b>  |
| 2.1      | DSU and Kruskal . . . . .             | 5         |
| 2.2      | Kahn topological sort . . . . .       | 5         |
| 2.3      | 0-1 BFS . . . . .                     | 5         |
| <b>3</b> | <b>A. Existing Network</b>            | <b>6</b>  |
| 3.1      | Restatement . . . . .                 | 6         |
| 3.2      | Constraints and traps . . . . .       | 6         |
| 3.3      | Observations . . . . .                | 6         |
| 3.4      | Sample walkthrough . . . . .          | 6         |
| 3.5      | Algorithm . . . . .                   | 7         |
| 3.6      | Correctness proof . . . . .           | 8         |
| 3.7      | Complexity . . . . .                  | 8         |
| 3.8      | C++ implementation notes . . . . .    | 8         |
| 3.9      | Python implementation notes . . . . . | 8         |
| 3.10     | Full C++ solution . . . . .           | 9         |
| 3.11     | Full Python solution . . . . .        | 10        |
| 3.12     | Common wrong answers . . . . .        | 11        |
| 3.13     | Tests that matter . . . . .           | 11        |
| <b>4</b> | <b>B. Build Timeline</b>              | <b>12</b> |
| 4.1      | Restatement . . . . .                 | 12        |
| 4.2      | Constraints and traps . . . . .       | 12        |
| 4.3      | Observations . . . . .                | 12        |
| 4.4      | Sample walkthrough . . . . .          | 12        |
| 4.5      | Algorithm . . . . .                   | 13        |
| 4.6      | Correctness proof . . . . .           | 14        |
| 4.7      | Complexity . . . . .                  | 14        |
| 4.8      | C++ implementation notes . . . . .    | 15        |
| 4.9      | Python implementation notes . . . . . | 15        |
| 4.10     | Full C++ solution . . . . .           | 15        |
| 4.11     | Full Python solution . . . . .        | 16        |
| 4.12     | Common wrong answers . . . . .        | 17        |
| 4.13     | Tests that matter . . . . .           | 17        |
| <b>5</b> | <b>C. One-Way Reversals</b>           | <b>18</b> |
| 5.1      | Restatement . . . . .                 | 18        |
| 5.2      | Constraints and traps . . . . .       | 18        |
| 5.3      | Observations . . . . .                | 18        |
| 5.4      | Sample walkthrough . . . . .          | 18        |
| 5.5      | Algorithm . . . . .                   | 19        |
| 5.6      | Correctness proof . . . . .           | 20        |

|          |                                       |           |
|----------|---------------------------------------|-----------|
| 5.7      | Complexity . . . . .                  | 20        |
| 5.8      | C++ implementation notes . . . . .    | 20        |
| 5.9      | Python implementation notes . . . . . | 21        |
| 5.10     | Full C++ solution . . . . .           | 21        |
| 5.11     | Full Python solution . . . . .        | 22        |
| 5.12     | Common wrong answers . . . . .        | 22        |
| 5.13     | Tests that matter . . . . .           | 23        |
| <b>6</b> | <b>Testing and stress strategy</b>    | <b>24</b> |
| <b>7</b> | <b>What to carry forward</b>          | <b>25</b> |

# 1 Editorial overview

This section is a recognition drill for the first graph phase.

| Problem              | Intended model                                 | Algorithm     |
|----------------------|------------------------------------------------|---------------|
| A. Existing Network  | free components plus paid undirected edges     | DSU + Kruskal |
| B. Build Timeline    | dependency DAG with longest prerequisite chain | Kahn + DP     |
| C. One-Way Reversals | directed roads become 0/1 weighted edges       | 0-1 BFS       |

The most important editorial habit is to separate:

```
modeling argument
algorithm correctness
implementation details
```

## 2 Tool reminders

### 2.1 DSU and Kruskal

DSU supports:

```
find(x): component representative
union(a,b): merge components if they are different
```

Kruskal:

```
sort edges by increasing weight
for each edge:
    if its endpoints are in different components:
        take it and union the components
```

The algorithm minimizes the total cost of connecting all components because at each cut between already-separated components, taking the cheapest safe edge is valid.

### 2.2 Kahn topological sort

Maintain indegrees. Vertices with indegree zero currently have no unmet prerequisites. Pop them, remove their outgoing edges, and push new indegree-zero vertices.

If the algorithm processes fewer than  $n$  vertices, some directed cycle remains.

### 2.3 0-1 BFS

For edge costs 0 and 1:

```
cost 0 relaxation -> push_front
cost 1 relaxation -> push_back
```

This maintains the same distance order that Dijkstra would produce for these restricted weights.

## 3 A. Existing Network

### 3.1 Restatement

There are  $n$  cities. Some undirected cables already exist and cost nothing. You may buy additional undirected cables, each with a nonnegative cost. Find the minimum total extra cost needed so all cities are connected. If no set of paid cables can connect all cities, print IMPOSSIBLE.

### 3.2 Constraints and traps

$n$ , free edges, and paid edges can each be as large as  $2 * 10^5$ , so an  $O(n^2)$  approach is impossible.

Important traps:

```
Free edges are real connections and must be merged first.  
The objective is to connect all cities, not to find a shortest route.  
The answer can exceed 32-bit int, so use long long / Python int.  
The graph may remain disconnected even after all paid edges are considered.
```

### 3.3 Observations

After using all free cables, the cities are partitioned into connected components. Inside one free component, no paid cable is needed.

So the problem becomes:

```
Given existing components, buy minimum-cost paid edges that connect them.
```

This is exactly Kruskal's MST idea, except that Kruskal starts after the free edges have already been unioned.

### 3.4 Sample walkthrough

Input:

```
5 2 4
1 2
4 5
2 3 4
3 4 7
1 5 20
2 5 10
```

Free unions:

```
{1,2}, {3}, {4,5}
```

Paid edges sorted:

```
2--3 cost 4
3--4 cost 7
2--5 cost 10
1--5 cost 20
```

Kruskal decisions:

```
take 2--3 cost 4    -> {1,2,3}, {4,5}
take 3--4 cost 7    -> {1,2,3,4,5}
stop
```

Answer:  $4 + 7 = 11$ .

## 3.5 Algorithm

```
read n, f, p
create DSU with n singleton components
components = n

for each free edge u, v:
    if union(u, v):
        components -= 1

read all paid edges as (weight, u, v)
sort paid edges by weight
answer = 0

for each paid edge in increasing weight:
    if components == 1:
        break
    if union(u, v):
        answer += weight
        components -= 1
```

```
if components == 1: print answer
else: print IMPOSSIBLE
```

### 3.6 Correctness proof

**Lemma 1.** After all free edges are unioned, each DSU component is exactly one connected component of the graph made only from free edges.

**Proof.** DSU union merges the endpoints of every free edge, so all vertices connected by a free-edge path end in the same DSU component. DSU never merges vertices except through a free edge, so vertices in different free connected components remain separated.  $\square$

**Lemma 2.** Running Kruskal on paid edges after the free unions produces a minimum-cost set of paid edges that connects the free components, if such a set exists.

**Proof.** Contract every free component into one super-vertex. A paid edge between two cities becomes an edge between their super-vertices; paid edges inside the same super-vertex are useless. Kruskal's algorithm on this contracted undirected graph returns a minimum spanning tree when the graph is connected. The DSU implementation after free unions is exactly Kruskal on that contracted graph.  $\square$

**Theorem.** The algorithm prints the minimum additional cost to connect all cities, or IMPOSSIBLE if it cannot be done.

**Proof.** By Lemma 1, the initial DSU state correctly represents the zero-cost connectivity. By Lemma 2, the paid edges selected afterward have minimum total cost among all ways to connect those components. If after processing all paid edges more than one component remains, no paid edge set can connect the contracted graph, so connecting all cities is impossible.  $\square$

### 3.7 Complexity

Sorting paid edges dominates:

```
Time:  $O((f + p) \alpha(n) + p \log p)$ 
Memory:  $O(n + p)$ 
```

Here  $\alpha(n)$  is the inverse Ackermann factor from DSU, effectively constant in contest constraints.

### 3.8 C++ implementation notes

Store paid edges as  $(\text{weight}, u, v)$  so normal tuple sorting sorts by weight first. Use `long long` for weights and total cost.

### 3.9 Python implementation notes

Use an iterative `find` with path compression. Python's integers are unbounded, so no special type is needed for the answer.

## 3.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, sz;
    DSU(int n) : parent(n), sz(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, f, p;
    if (!(cin >> n >> f >> p)) return 0;
    DSU dsu(n);
    int components = n;
    for (int i = 0; i < f; i++) {
        int u, v;
        cin >> u >> v;
        if (dsu.unite(u - 1, v - 1)) components--;
    }
    vector<tuple<long long, int, int>> edges;
    for (int i = 0; i < p; i++) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        edges.push_back({w, u - 1, v - 1});
    }
    sort(edges.begin(), edges.end());
    long long total = 0;
```

```

for (auto [w, u, v] : edges) {
    if (components == 1) break;
    if (dsu.unite(u, v)) {
        total += w;
        components--;
    }
}
if (components == 1) cout << total << '\n';
else cout << "IMPOSSIBLE\n";
return 0;
}

```

### 3.11 Full Python solution

```

import sys

class DSU:
    def __init__(self, n: int) -> None:
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a: int, b: int) -> bool:
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return True

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    f = next(it)
    p = next(it)

```

```

dsu = DSU(n)
components = n
for _ in range(f):
    u = next(it) - 1
    v = next(it) - 1
    if dsu.union(u, v):
        components -= 1
edges = []
for _ in range(p):
    u = next(it) - 1
    v = next(it) - 1
    w = next(it)
    edges.append((w, u, v))
edges.sort()
total = 0
for w, u, v in edges:
    if components == 1:
        break
    if dsu.union(u, v):
        total += w
        components -= 1
print(total if components == 1 else "IMPOSSIBLE")

if __name__ == "__main__":
    main()

```

### 3.12 Common wrong answers

Sorting all edges but forgetting to include free connectivity first.  
 Adding a paid edge even when both endpoints are already in the same component.  
 Printing the partial cost when the graph remains disconnected.  
 Using int for the total cost in C++.

### 3.13 Tests that matter

Already connected using only free edges -> answer 0.  
 No paid edge can connect all components -> IMPOSSIBLE.  
 Multiple paid edges between same components -> choose the cheapest useful one.  
 Large weights -> verifies long long.

## 4 B. Build Timeline

### 4.1 Restatement

There are  $n$  modules. Module  $i$  takes  $\text{time}[i]$  days. A dependency  $a \ b$  means module  $a$  must finish before module  $b$  may start. For each query module, print the earliest day it can finish. If the dependency graph contains a cycle, print IMPOSSIBLE.

### 4.2 Constraints and traps

$n$ ,  $m$ , and  $q$  can be up to  $2 * 10^5$ , so we need linear graph processing.

Important traps:

```
A dependency edge is directed.
Cycles make the whole schedule impossible.
Earliest start is the maximum finish time among prerequisites.
Many queries share the same graph, so compute all finish times once.
Finish times can exceed int.
```

### 4.3 Observations

If there is a cycle, every module on that cycle waits, directly or indirectly, for itself. No valid schedule exists.

If the graph is acyclic, process modules in topological order. When module  $u$  is processed, its earliest finish time is known. For every edge  $u \rightarrow v$ ,  $v$  cannot start before  $\text{finish}[u]$ , so:

```
start[v] = max(start[v], finish[u])
```

After all incoming edges of  $v$  have been processed,  $\text{start}[v]$  contains the maximum finish time of all prerequisites.

### 4.4 Sample walkthrough

Input:

```
5 5 3
3 2 4 6 1
1 3
2 3
3 4
2 5
5 4
3 4 5
```

Initial zero-indegree modules:

```
1 and 2
```

Process:

```
finish[1] = 3
  start[3] = max(0, 3) = 3

finish[2] = 2
  start[3] = max(3, 2) = 3
  start[5] = max(0, 2) = 2

finish[3] = 3 + 4 = 7
  start[4] = max(0, 7) = 7

finish[5] = 2 + 1 = 3
  start[4] = max(7, 3) = 7

finish[4] = 7 + 6 = 13
```

Queries 3 4 5 give:

```
7 13 3
```

## 4.5 Algorithm

```
read n, m, q
read duration[1..n]
build adjacency list and indegree array
read queries

queue = all vertices with indegree 0
start[v] = 0 for all v
processed = 0

while queue not empty:
  u = pop queue
  processed += 1
```

```

    finish[u] = start[u] + duration[u]
    for each u -> v:
        start[v] = max(start[v], finish[u])
        indegree[v] -= 1
        if indegree[v] == 0:
            push v

if processed < n:
    print IMPOSSIBLE
else:
    print finish[x] for each query x

```

## 4.6 Correctness proof

**Lemma 1.** Every vertex popped by Kahn’s algorithm is popped only after all its prerequisites have been popped.

**Proof.** A vertex enters the queue only when its indegree becomes zero. Its indegree is decremented once for each incoming edge after that predecessor is popped. Therefore all incoming neighbors have already been processed before the vertex is popped.  $\square$

**Lemma 2.** When vertex  $v$  is popped,  $\text{start}[v]$  equals the earliest possible start time of module  $v$ .

**Proof.** By Lemma 1, all prerequisites of  $v$  have already been processed. Each prerequisite  $u$  updated  $\text{start}[v]$  with  $\text{finish}[u]$ , so  $\text{start}[v]$  is at least the finish time of every prerequisite. The algorithm uses the maximum of those values, which is exactly the earliest time when all prerequisites have finished.  $\square$

**Lemma 3.** If Kahn’s algorithm processes fewer than  $n$  vertices, the graph contains a directed cycle.

**Proof.** If processing stops early, every remaining vertex has positive indegree within the remaining subgraph. Starting from any remaining vertex and following an incoming edge forever must eventually repeat a vertex, forming a directed cycle.  $\square$

**Theorem.** The algorithm prints IMPOSSIBLE exactly for cyclic dependency graphs; otherwise it prints the earliest finish time for each queried module.

**Proof.** By Lemma 3, an early stop identifies a directed cycle, which makes a valid schedule impossible. If all vertices are processed, Lemma 2 gives the earliest start time for each module when it is popped, so  $\text{finish}[v] = \text{start}[v] + \text{duration}[v]$  is the earliest finish time. Queries read from these values.  $\square$

## 4.7 Complexity

```

Time:   $O(n + m + q)$ 
Memory:  $O(n + m + q)$ 

```

## 4.8 C++ implementation notes

Use `queue<int>` for Kahn's algorithm. Store `duration`, `start`, and `finish` as `long long`.

## 4.9 Python implementation notes

Use `collections.deque` as the queue. Reading the entire input at once keeps the parser fast enough for the largest cases.

## 4.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, qn;
    if (!(cin >> n >> m >> qn)) return 0;
    vector<long long> duration(n);
    for (long long &x : duration) cin >> x;
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a - 1].push_back(b - 1);
        indeg[b - 1]++;
    }
    vector<int> queries(qn);
    for (int &x : queries) {
        cin >> x;
        --x;
    }
    vector<long long> start(n, 0), finish = duration;
    queue<int> q;
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
    int processed = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        processed++;
        finish[u] = start[u] + duration[u];
        for (int v : adj[u]) {
            start[v] = max(start[v], finish[u]);
            indeg[v]--;
        }
    }
```

```

        if (indeg[v] == 0) q.push(v);
    }
}
if (processed != n) {
    cout << "IMPOSSIBLE\n";
} else {
    for (int i = 0; i < qn; i++) {
        if (i) cout << ' ';
        cout << finish[queries[i]];
    }
    cout << '\n';
}
return 0;
}

```

## 4.11 Full Python solution

```

from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    qn = next(it)
    duration = [next(it) for _ in range(n)]
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for _ in range(m):
        a = next(it) - 1
        b = next(it) - 1
        adj[a].append(b)
        indeg[b] += 1
    queries = [next(it) - 1 for _ in range(qn)]
    start = [0] * n
    finish = duration[:]
    dq = deque(i for i in range(n) if indeg[i] == 0)
    processed = 0
    while dq:
        u = dq.popleft()
        processed += 1
        finish[u] = start[u] + duration[u]
        for v in adj[u]:
            if finish[u] > start[v]:

```

```

        start[v] = finish[u]
        indeg[v] -= 1
        if indeg[v] == 0:
            dq.append(v)
    if processed != n:
        print("IMPOSSIBLE")
    else:
        print(" ".join(str(finish[x]) for x in queries))

if __name__ == "__main__":
    main()

```

## 4.12 Common wrong answers

Using DFS reachability instead of topological sorting.  
 Updating start[v] with min instead of max.  
 Answering queries before detecting cycles.  
 Forgetting that independent modules can start at day 0.  
 Using int for long timelines in C++.

## 4.13 Tests that matter

No dependencies: finish[i] = duration[i].  
 A simple chain: finish accumulates all previous durations.  
 A diamond dependency: the join node waits for the slower branch.  
 A directed cycle: output IMPOSSIBLE.  
 A cycle disconnected from queried modules: still IMPOSSIBLE, because the problem says the dependencies contain a cycle.

## 5 C. One-Way Reversals

### 5.1 Restatement

There are directed roads. Traveling along a road's given direction costs nothing. Traveling against a road is allowed, but costs 1 reversal for that trip. Find the minimum number of reversals needed to travel from city 1 to city  $n$ , or print -1 if there is no possible route even after reversals.

### 5.2 Constraints and traps

$n$  and  $m$  can be up to  $2 * 10^5$ , so use an adjacency list and near-linear shortest path.

Important traps:

```
The original roads are directed.
Each input road creates two possible travel edges with different costs.
Ordinary BFS is wrong because cost 0 and cost 1 moves are not equal.
Dijkstra is correct but heavier than needed; 0-1 BFS is the intended tool.
The answer is -1 only if city  $n$  is unreachable in the underlying undirected
graph formed by the roads.
```

### 5.3 Observations

For every input road:

```
u -> v
```

we add:

```
u -> v cost 0
v -> u cost 1
```

Now every route in the original problem corresponds to a path in this weighted graph. The path cost is exactly the number of reversed roads used. Because all costs are 0 or 1, use 0-1 BFS.

### 5.4 Sample walkthrough

Input:

```
5 5
1 2
3 2
3 4
5 4
2 5
```

The route:

```
1 -> 2 -> 5
```

uses both roads in their original direction:

```
1 -> 2 cost 0
2 -> 5 cost 0
```

So the answer is 0.

Another small case:

```
3 2
2 1
3 2
```

To go from 1 to 3:

```
1 -> 2 reverses 2 -> 1, cost 1
2 -> 3 reverses 3 -> 2, cost 1
```

Answer: 2.

## 5.5 Algorithm

```
read n, m
adj = empty weighted adjacency list

for each road u, v:
    add edge u -> v with cost 0
    add edge v -> u with cost 1

dist[1] = 0
deque = [1]

while deque not empty:
    u = pop front
    for each edge u -> v with cost w:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
```

```

        if w == 0: push_front(v)
        else:      push_back(v)

if dist[n] is infinity: print -1
else: print dist[n]

```

## 5.6 Correctness proof

**Lemma 1.** Every valid trip in the original problem corresponds to a path in the constructed graph with the same number of reversals.

**Proof.** For each traveled road, if the trip follows the road direction, the constructed graph contains the cost-0 edge. If the trip goes against the road, the constructed graph contains the reverse cost-1 edge. Summing costs counts exactly the reversed roads.  $\square$

**Lemma 2.** Every path in the constructed graph corresponds to a valid trip in the original problem with the same reversal count.

**Proof.** Every constructed edge came from an original road. A cost-0 edge uses that road in its given direction; a cost-1 edge uses it against its direction and pays one reversal. Thus the graph path is a valid trip with equal cost.  $\square$

**Lemma 3.** 0-1 BFS computes shortest distances in the constructed graph.

**Proof.** When relaxing a cost-0 edge, the new state has the same distance as the current state, so it belongs at the front of the deque. When relaxing a cost-1 edge, the new state is one distance layer later, so it belongs at the back. This preserves nondecreasing distance processing, matching Dijkstra's order for edge weights 0 and 1.  $\square$

**Theorem.** The algorithm prints the minimum number of reversals needed to travel from city 1 to city  $n$ , or  $-1$  if no valid trip exists.

**Proof.** By Lemmas 1 and 2, minimizing reversals in the original problem is equivalent to finding a shortest path in the constructed 0/1 graph. By Lemma 3, 0-1 BFS computes that shortest distance. If the distance remains infinity, there is no path in the constructed graph, hence no valid trip.  $\square$

## 5.7 Complexity

Each original road creates two adjacency entries.

```

Time:   $O(n + m)$ 
Memory:  $O(n + m)$ 

```

## 5.8 C++ implementation notes

Use `deque<int>` and adjacency entries `(to, cost)`. The maximum answer is at most  $m$ , so `int` distances are enough here.

## 5.9 Python implementation notes

Use `collections.deque`. The graph has  $2m$  stored edges, so append tuples carefully but directly; no heap is needed.

## 5.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<pair<int,int>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        adj[u].push_back({v, 0});
        adj[v].push_back({u, 1});
    }
    const int INF = 1e9;
    vector<int> dist(n, INF);
    deque<int> dq;
    dist[0] = 0;
    dq.push_back(0);
    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto [v, w] : adj[u]) {
            int nd = dist[u] + w;
            if (nd < dist[v]) {
                dist[v] = nd;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    cout << (dist[n - 1] == INF ? -1 : dist[n - 1]) << '\n';
    return 0;
}
```

## 5.11 Full Python solution

```
from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append((v, 0))
        adj[v].append((u, 1))
    inf = 10**9
    dist = [inf] * n
    dist[0] = 0
    dq = deque([0])
    while dq:
        u = dq.popleft()
        for v, w in adj[u]:
            nd = dist[u] + w
            if nd < dist[v]:
                dist[v] = nd
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    print(-1 if dist[n - 1] == inf else dist[n - 1])

if __name__ == "__main__":
    main()
```

## 5.12 Common wrong answers

Adding only the original directed cost-0 edges.  
Adding reverse edges with cost 0.  
Using BFS and counting number of roads instead of reversals.  
Printing 0 for unreachable cities.  
Using push\_back for both cost 0 and cost 1 relaxations.

## 5.13 Tests that matter

There is already a directed path from 1 to n -> answer 0.

All roads on the only path point backward -> answer equals path length.

City n is isolated -> -1.

There is a longer 0-cost route and a shorter 1-cost route -> choose 0-cost.

Parallel roads or repeated relaxations -> distance updates still work.

## 6 Testing and stress strategy

The section judge uses:

```
CP_TARGET=solution python3 sections/21_graphs_i_mixed_contest/check.py
```

The random tests target different risks:

Existing Network:

compare Kruskal behavior on connected, disconnected, already-connected, and duplicate-edge cases.

Build Timeline:

generate both DAGs and cyclic graphs; check no-edge and single-node cases.

One-Way Reversals:

compare 0-1 BFS answers on random directed graphs, including unreachable targets.

For personal stress testing, a brute force oracle is feasible on tiny graphs:

A: enumerate subsets of paid edges for  $n \leq 8$ .

B: for DAGs, compare against repeated relaxation in topological order; for cycles, compare against DFS cycle detection.

C: run ordinary Dijkstra on the constructed 0/1 graph and compare to 0-1 BFS.

## 7 What to carry forward

Mixed graph problems are mostly about modeling discipline.

```
Connect all vertices cheaply -> MST thinking.  
Respect prerequisites -> topological thinking.  
Minimize travel cost -> inspect edge weights before choosing BFS, 0-1 BFS, or  
Dijkstra.
```

In future sections, graph problems will combine these tools with dynamic programming, trees, and harder state spaces. The same diagnosis routine remains the first step.